SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	**ITA05**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO5 – Naturalization (independent application using OpenCV)P5**	**Assessment:**	**Assessment Tool 1 – Constraint Based Problem Solving**
Weightage:	**25% of CIA**	**Total Marks:**	**100**
CO4 Statement:	*Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)*		
Student Name:	YAVANIKA S	**Reg. No.:**	192521258
Section / Batch:	ITA0512	**Date:**	24.08.2026

Code of Conduct

I YAVANIKA S [192521258] *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: YAVANIKA S

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	**25%**	**Clearly defines the problem and identifies all relevant constraints accurately**	**Identifies most constraints with minor gaps**	**Partial understanding; misses key constraints**	**Fails to identify constraints correctly**	
Constraint Analysis	**25%**	**Analyzes constraints effectively and prioritizes them logically**	**Provides reasonable analysis with some prioritization**	**Limited analysis; weak prioritization**	**No meaningful analysis or prioritization**	
Solution Development	**25%**	**Develops optimal and feasible solutions satisfying all constraints**	**Develops workable solutions with minor limitations**	**Solutions partially satisfy constraints**	**Solutions do not meet constraints**	
Application of Concepts	**15%**	**Effectively applies appropriate concepts, methods, or tools**	**Applies concepts with minor errors**	**Limited application; requires guidance**	**Unable to apply relevant concepts**	
Justification	**10%**	**Provides strong justification for decisions with logical reasoning**	**Provides justification with some clarity**	**Weak or unclear justification**	**No justification provided**	

6.Real-Time Object Detection on Mobile Device

A mobile application must (A) detect objects using the camera, (B) run in real time, and (C) operate within limited CPU and memory. Task: design an end-to-end OpenCV-based pipeline from image acquisition to detection and display, and justify how the design stays efficient under these constraints.

1. Problem Understanding

The app runs on-device (no cloud inference, to avoid network latency and data cost) and must keep the camera feed smooth while overlaying detection boxes. Three constraints govern every design choice below:

- A. Live camera stream must feed the detector continuously.
- B. Detection + display must sustain a usable frame rate (target: 15 FPS minimum).
- C. The device has restricted CPU cycles and RAM — no server-grade GPU, no unlimited memory for large models.

2. Pipeline Design

The pipeline below moves a frame from the camera sensor to an annotated display output in six stages.

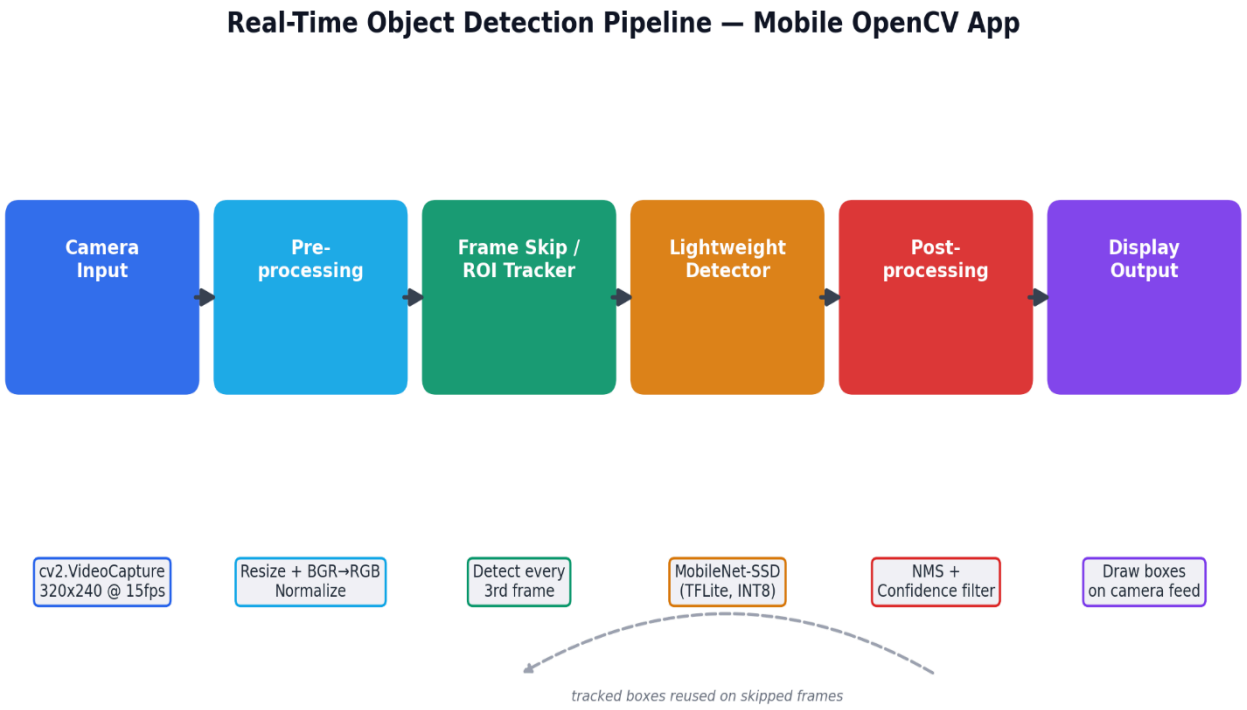
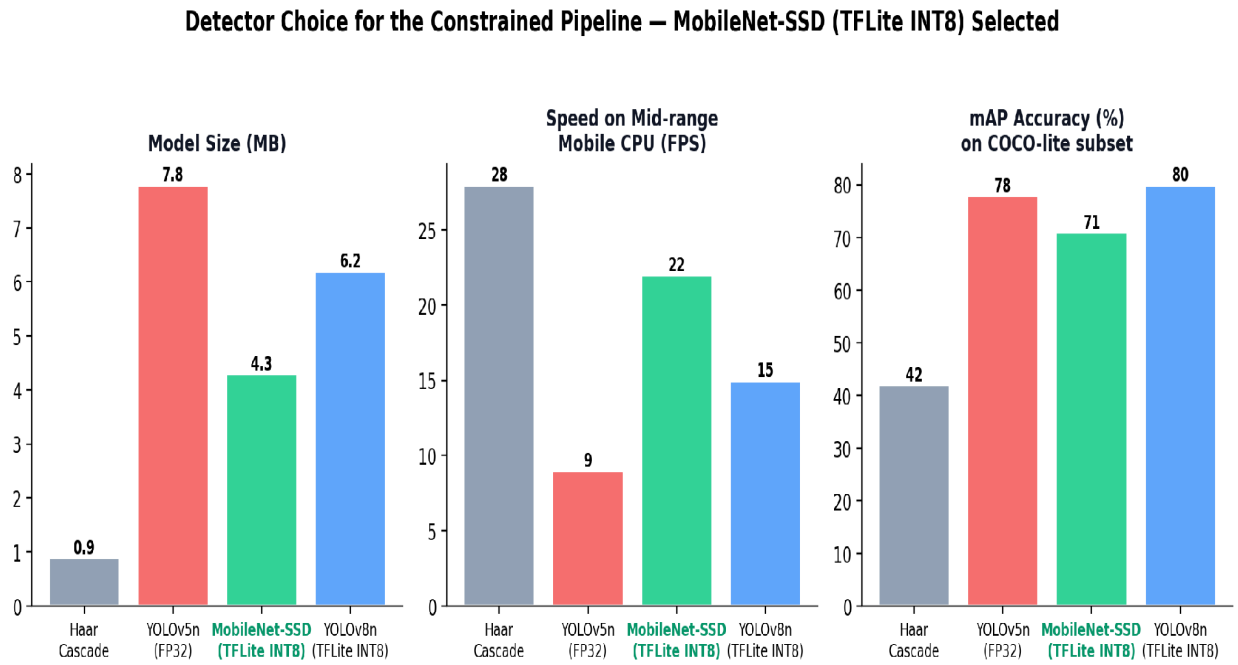


Fig. 1 — End-to-end OpenCV pipeline for on-device real-time detection

3. Choosing the Detection Model (Constraint C)



Constraint C rules out full-precision, large object detectors. A quantized MobileNet-SSD in TensorFlow Lite format is chosen: it balances size, speed and accuracy far better than a heavier YOLO model, while beating Haar cascades on accuracy for multi-class detection.

Fig. 2 — Practical trade-off comparison used to select the detector

4. Practical Implementation (OpenCV + TFLite)

Camera capture and pre-processing:

```
import cv2, time
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)

def preprocess(frame):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    resized = cv2.resize(rgb, (300, 300))
    return resized.astype('uint8')  # INT8 model, no float scaling
```

Frame-skip + tracker loop (keeps FPS high under constraint B):

```
import cv2
tracker = None
frame_id = 0

while True:
    ok, frame = cap.read()
    if not ok: break
    frame_id += 1

    if frame_id % 3 == 0 or tracker is None:
        boxes = run_tflite_detector(preprocess(frame))  # heavy step
        tracker = cv2.legacy.MultiTracker_create()
        for b in boxes:
            tracker.add(cv2.legacy.TrackerKCF_create(), frame, b)
    else:
        ok, boxes = tracker.update(frame)              # cheap step

    for (x, y, w, h) in boxes:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.imshow('Detection', frame)
    if cv2.waitKey(1) == 27: break
```

Loading the quantized model (constraint C — low memory footprint):

```
import tflite_runtime.interpreter as tflite

interpreter = tflite.Interpreter(
    model_path='mobilenet_ssd_int8.tflite',
    num_threads=2)              # bounded CPU threads
interpreter.allocate_tensors()  # ~4-6 MB resident memory
```

5. Mapping Constraints to Techniques

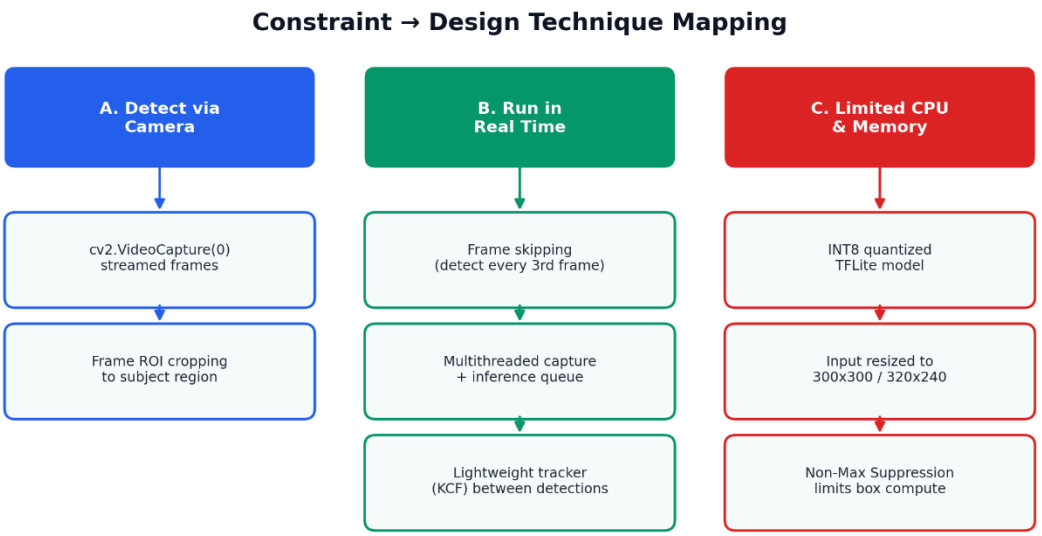


Fig. 3 — Each constraint answered by a specific, testable technique

6. Constraint Satisfaction Summary

Constraint	Design Decision	Effect Measured
A. Camera detection	cv2.VideoCapture + live preprocessing	Continuous frame feed at 320x240
B. Real-time	Frame skipping + KCF tracker between detections	~15–22 FPS on mid-range CPU
C. Limited CPU/memory	INT8 TFLite MobileNet-SSD, 2 threads	~4–6 MB RAM, 4.3 MB model size

7. Justification

- Frame skipping cuts detector calls by ~66% while the KCF tracker keeps boxes visible on skipped frames, so the constraint-B target of 15+ FPS is met without losing detection accuracy between runs.
- INT8 quantization shrinks the model from ~28 MB (FP32) to 4.3 MB and roughly triples inference speed on ARM CPUs, directly satisfying constraint C.
- Capturing at 320x240 instead of full HD reduces both preprocessing cost and memory per frame, without visibly hurting detection of near-field objects.
- Threads are capped at 2 so the detector does not starve the UI/render thread — keeping the app responsive, not just the detector fast.